

Activités : collaborateur_id et Swagger

1. Objectif et film mental

Visualiser le parcours POST, GET et PATCH : choisir un collaborateur actif du projet puis retirer son affectation avec null. Attention au contrat, engagement par les exemples, retour des tests, consolidation par la reformulation.

2. Architecture et invariants

collaborateur_id conserve un seul I. La ForeignKey collaborateur référence accounts.Utilisateur avec SET_NULL. Le champ reste nullable et facultatif. Les anciennes dépendances restent en base sans conversion de leurs identifiants. equipe_ids est indépendant.

3. Guide pratique

Ajouter la ForeignKey nullable au modèle. Déclarer

PrimaryKeyRelatedField(source="collaborateur") dans les serializers. En lecture : read_only=True et allow_null=True. En écriture : queryset des utilisateurs actifs et non supprimés, required=False et allow_null=True. Remplacer dependance dans Meta.fields. Valider les affectations actives au projet, son chef de projet et son conducteur de travaux. Le PATCH omettant le champ conserve la relation, null la retire.

4. Migration après synchronisation develop

La migration finale est 0027_activite_collaborateur.py, dépendant de 0026_projet_badge_sante_arretchantier_santeprojetsnapshot. Elle ajoute le champ sans supprimer les données historiques. Vérifier le graphe puis appliquer python manage.py migrate_schemas sur un environnement sauvegardé. Les anciennes dépendances gardent leurs protections de suppression.

5. Swagger et contrats

Les schémas POST ActiviteCreation, PATCH PatchedActiviteModification et GET Activite exposent un UUID nullable. Le GET est en lecture seule. dependance est absent des propriétés et des exemples. Actualiser les descriptions extend_schema et vérifier le JSON généré par SchemaGenerator(config.urls_tenant). Le frontend doit adopter ce nouveau contrat.

6. Signal d'erreur et pré-mortem

400 pour UUID invalide, utilisateur absent, inactif, supprimé ou non affecté. Le champ dependance requis est rejeté. Les droits d'accès au projet restent nécessaires. Ne pas confondre UUID activité et utilisateur. Les hypothèses se valident avec des tests observables, sans forcer la conclusion.

7. Tests et transmission

62 tests validation et Swagger passent après intégration distante. Le graphe de migrations est sans conflit et cohérent avec les modèles. Les tests PostgreSQL n'ont pas pu aboutir car localhost:5432 est inaccessible. Relancer pytest apps/projets/tests/test_activites_formulaire.py apps/projets/tests/test_lots_activites_mutations.py -q lorsque PostgreSQL est prêt. Expliquez ? un pair le rôle de source et pourquoi une dépendance ne se transforme pas par renommage en

utilisateur.

```
{"libelle": "Terrassement",  
  "colaborateur_id": "<UUID utilisateur>"}  
PATCH : {"colaborateur_id": null}
```